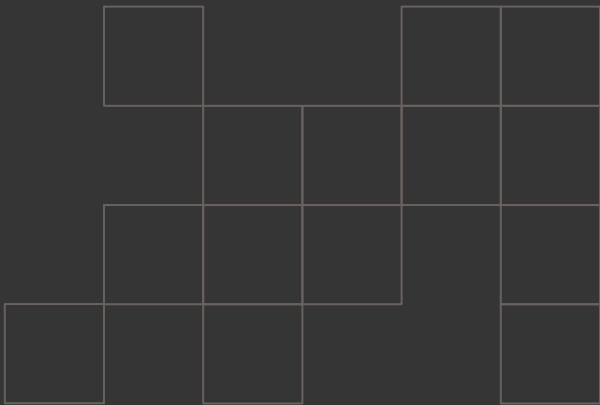


2026-01-14 - Lightning talk

Go beyond channels

samber/ro



whoami

My name is Samuel

- > Ex-CTO @ Screeb, SWE
- > GenAI Nantes & Shift organizer
- > Full-time open-sourcer in Q4 2025
- > Open to work (2026)

github.com/samber



My open-source projects



Samuel Berthe

samber · he/him

Software Engineer | Go Open-sourcer |
GenAI community leader

Pinned

[Customize your pins](#)

 **lo** Public

🔥 A Lodash-style Go library based on Go 1.18+ Generics (map, filter, contains, find...)

Go ⭐ 20.8k 🍴 926

 **do** Public

⚙️ A dependency injection toolkit based on Go 1.18+ Generics.

Go ⭐ 2.6k 🍴 103

 **mo** Public

🔥 Monads and popular FP abstractions, powered by Go 1.18+ Generics (Option, Result, Either...)

Go ⭐ 3.3k 🍴 109

 **ro** Public

🔥 Streams & Reactive Programming paradigm for Go: declarative and composable API for event-driven applications

Go ⭐ 564 🍴 17

 **awesome-prometheus-alerts** Public

🔥 Collection of Prometheus alerting rules

HTML ⭐ 7.7k 🍴 1.1k

 **oops** Public

🔥 Error handling library with context, assertion, stack trace and source fragments

Go ⭐ 839 🍴 34

```

rand.Seed(time.Now().UnixNano())

source := make(chan int)
processed := make(chan int)
done := make(chan struct{})

// Producer
go func() {
    for i := 0; i < 10; i++ {
        val := rand.Intn(100)
        fmt.Printf("producer: %d\n", val)
        source <- val
    }
    close(source)
}()

// Workers
var wg sync.WaitGroup
for w := 0; w < 10; w++ {
    wg.Add(1)
    go func(id int) {
        defer wg.Done()
        for v := range source {
            res := v * 2
            fmt.Printf("worker %d: %d -> %d\n", id, v, res)
            processed <- res
        }
    }(w)
}

```

```

// Close processed channel after all workers finish
go func() {
    wg.Wait()
    close(processed)
}()

// Consumer
go func() {
    for v := range processed {
        fmt.Printf("collector: received %d\n", v)
    }
    done <- struct{}{}
}()

<-done

```

A multi-stage pipeline of data transformation.

Not readable 🤔

A channel ?

Use cases:

- 1-way communication between goroutines

```
9
10 func main() {
11     ch := make(chan int)
12
13     go func() {
14         for i := range 42 {
15             ch <- i
16         }
17     }()
18
19     go func() {
20         for result := range ch {
21             fmt.Println(result)
22         }
23     }()
24 }
25
```

A channel ?

Use cases:

- 1-way communication between goroutines
- Queues (using bounded channels)

```
10 func main() {  
11     ch := make(chan int, 42)  
12     ch <- 2  
13     ch <- 4  
14     ch <- 8  
15     ch <- 10  
16  
17     go func() {  
18         for result := range ch {  
19             fmt.Println(result)  
20         }  
21     }()  
22 }  
23
```

A channel ?

Use cases:

- 1-way communication between goroutines
- Queues (using bounded channels)
- Sleep or wake-up multiple concurrent tasks

```
9
10 func main() {
11     done := make(chan struct{})
12
13     go func() {
14         // very long task 1
15         done <- struct{}{}
16     }()
17     go func() {
18         // very long task 2
19         done <- struct{}{}
20     }()
21
22     <-done
23     <-done
24     close(done)
25 }
```

A channel ?

Low level features: thread safe

```
9
10 func main() {
11     ch := make(chan int, 42)
12
13     go func() {
14         ch<-42
15     }()
16
17     go func() {
18         ch<-42
19     }()
20 }
21
```


A channel ?

Low level features: blocking

```
9
10 func main() {
11     ch := make(chan int)
12
13     go func() {
14         ch<-42 // blocks until the consumer receives data
15     }()
16
17     result := <-ch // blocks until the producer sends data
18 }
19
```

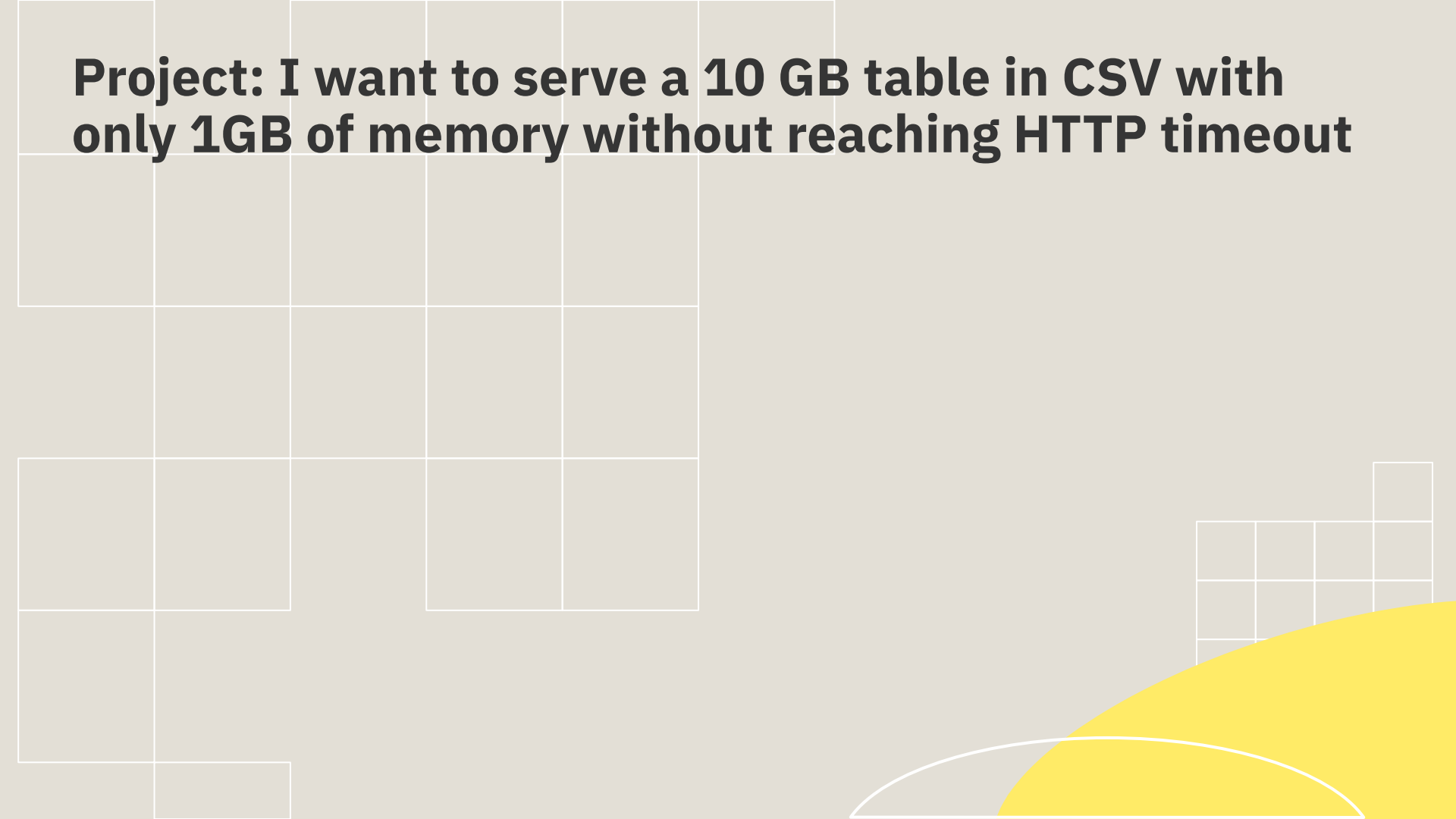
A channel ?

Low level features: buffer

```
9
10 func main() {
11     ch := make(chan int) // unbuffered
12     ch <- 42              // blocks until another goroutines runs `<-ch`
13 }
14
```

```
9
10 func main() {
11     ch := make(chan int, 42) // buffered
12     ch <- 42                  // not blocking except if buffer is full
13 }
14
```

Project: I want to serve a 10 GB table in CSV with only 1GB of memory without reaching HTTP timeout

The background features a light gray grid of squares. On the left side, there are several squares of varying sizes, some of which are outlined in white. On the right side, there is a yellow curved shape that resembles a hill or a wave, also outlined in white. The text is positioned in the upper left area of the image.

Project: I want to serve a 10 GB table in CSV with only 1GB of memory without reaching HTTP timeout

Using slice in my pipeline:

- ❌ Async & event driven
- ✅ Order is preserved
- ❌ Unknown length of items
- ⚠️ Batching with buffers
- ❌ No support for retry
- ✅ I don't need thread safety
- ✅ Fast

Project: I want to serve a 10 GB table in CSV with only 1GB of memory without reaching HTTP timeout

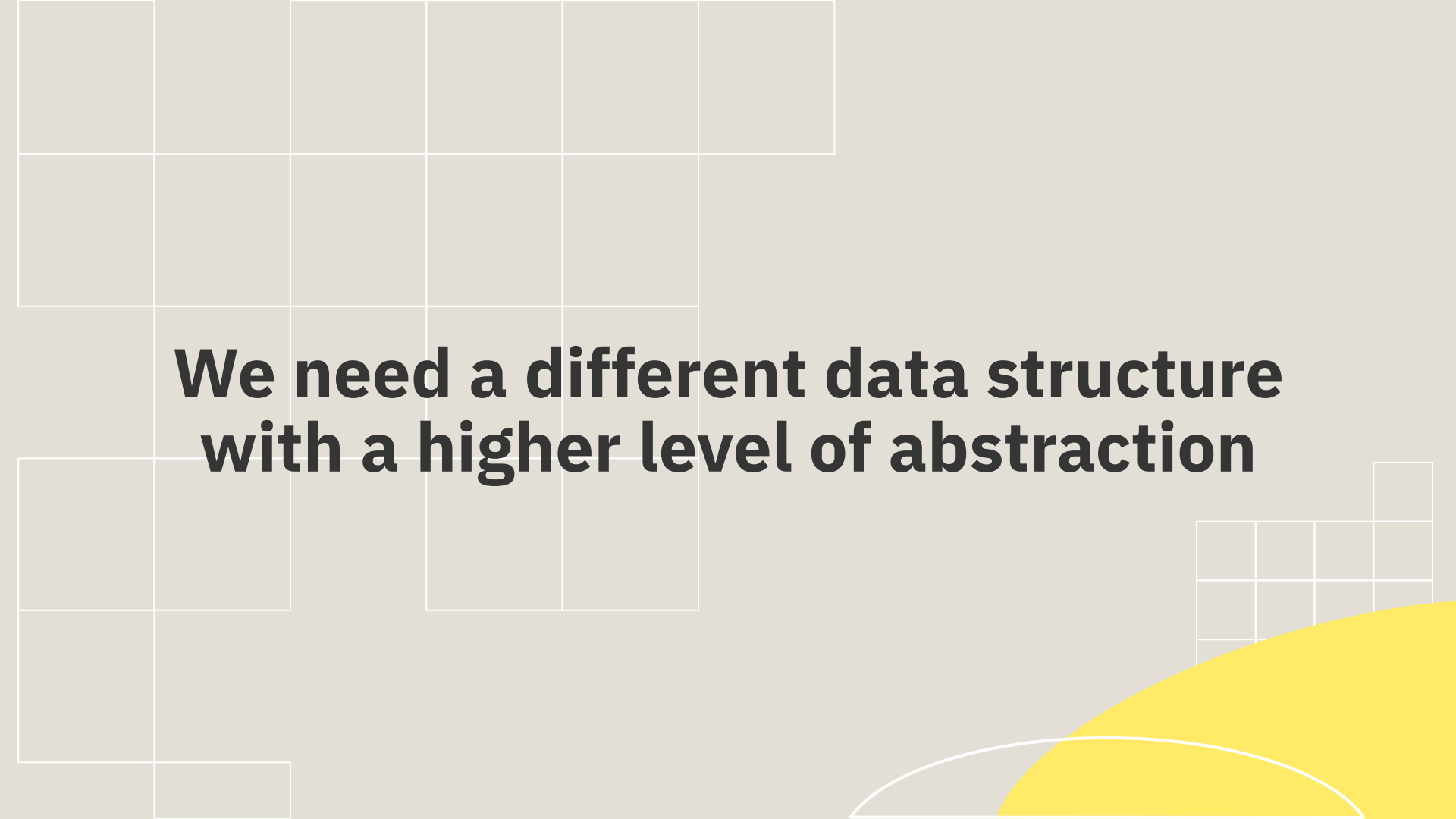
Using channels in my pipeline:

-  Async & event driven
-  Order is preserved
-  Unknown length of items
-  Batching with buffers
-  No support for retry
-  I don't need thread safety
-  Fast

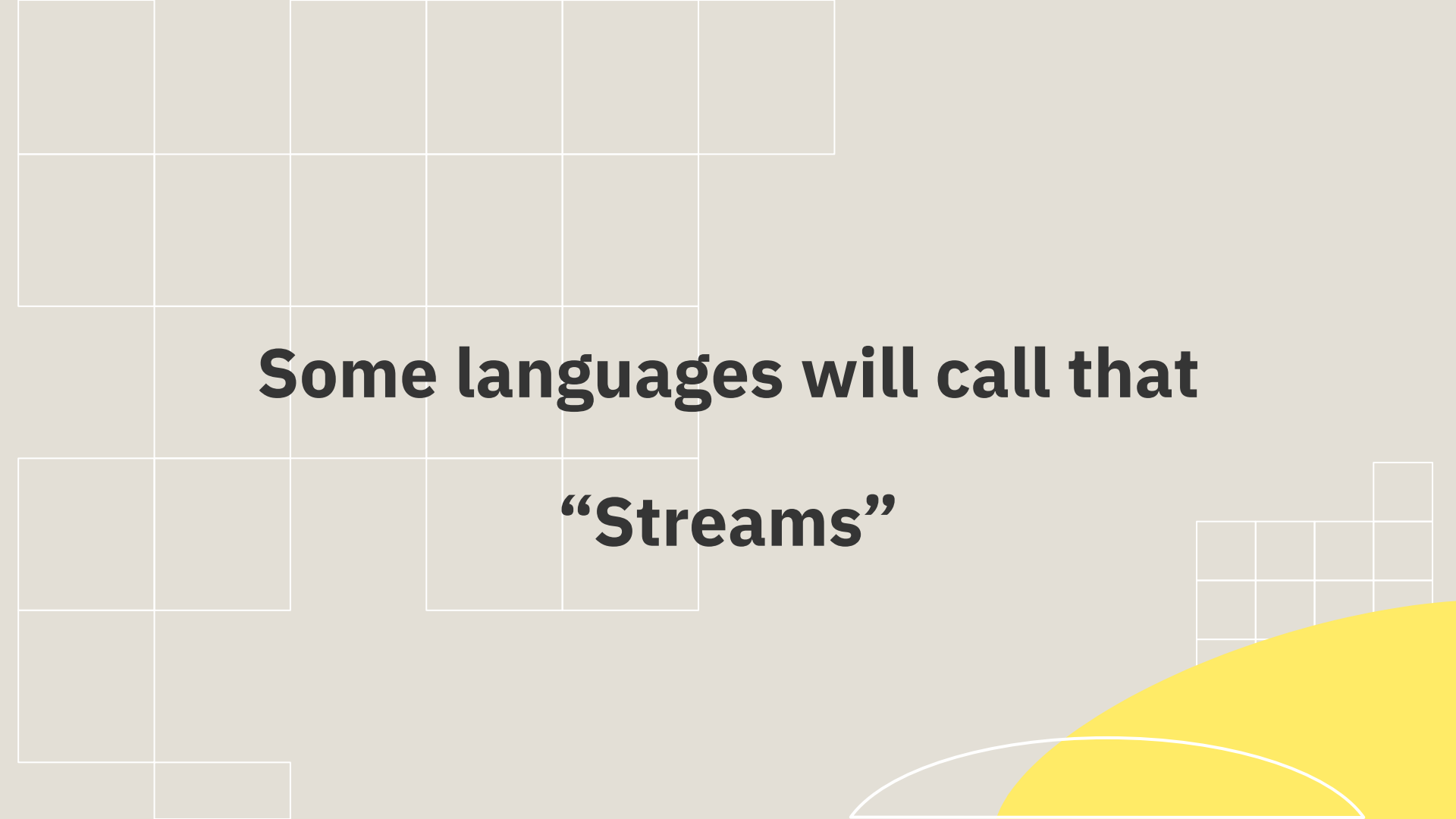
Project: I want to serve a 10 GB table in CSV with only 1GB of memory without reaching HTTP timeout

I want the following properties:

-  Async & event driven
-  Order is preserved
-  Unknown length of items
-  Batching with buffers
-  Support for retry
-  No thread safety
-  Fast

The background is a light gray color. It features several white-outlined squares of varying sizes arranged in a grid-like pattern. In the bottom right corner, there is a large, solid yellow curved shape that resembles a stylized hill or a large arc. The text is centered in the middle of the image.

**We need a different data structure
with a higher level of abstraction**



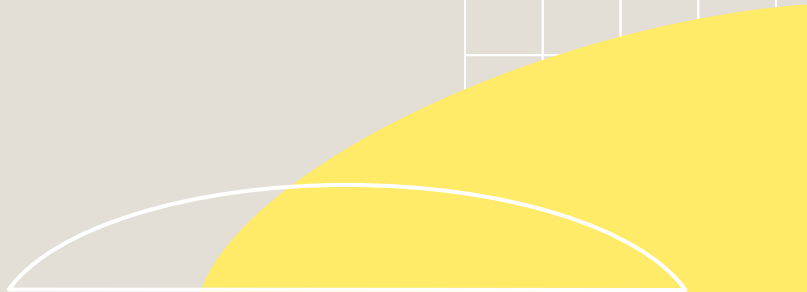
Some languages will call that

“Streams”

A decorative background featuring a light gray grid pattern. The grid is composed of squares of varying sizes, some of which are slightly offset or missing, creating a fragmented, architectural look. The text is centered over this grid.

Let me introduce

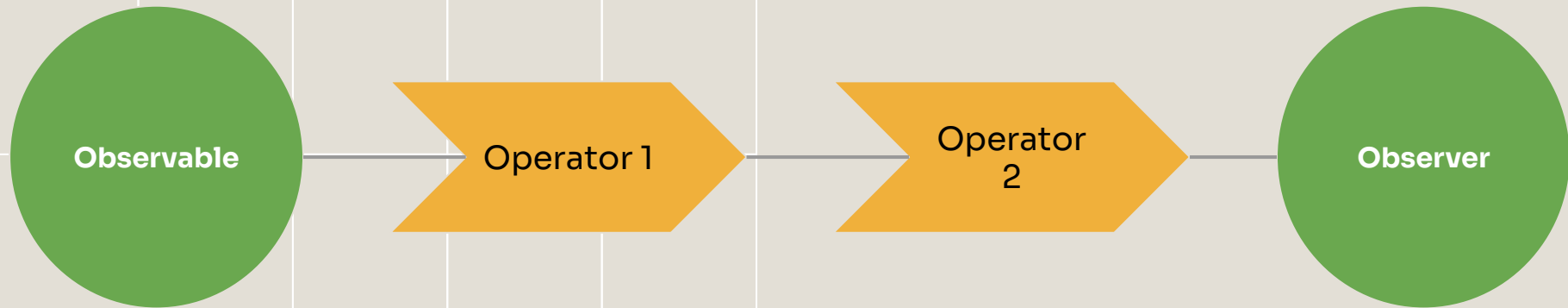
The “Reactive Programming” paradigm

A bright yellow, curved shape resembling a stylized sun or a rising horizon, located in the bottom right corner of the slide. It has a white outline and a slight gradient.

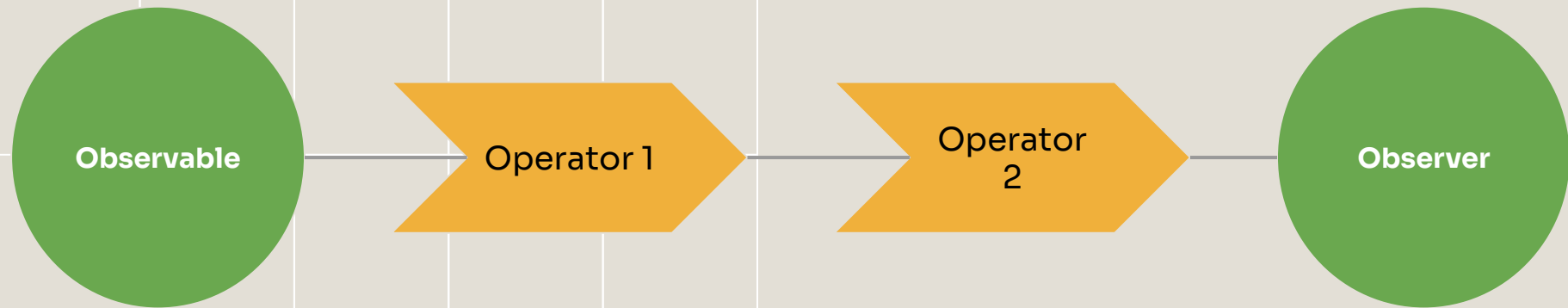
Hello world

```
observable := ro.Pipe(  
    ro.RangeWithInterval(0, 5, 1*time.Second),  
    ro.Filter(func(x int) bool {  
        return x%2 == 0  
    }),  
    ro.Map(func(x int) string {  
        return fmt.Sprintf("even-%d", x)  
    }),  
)
```

Hello world



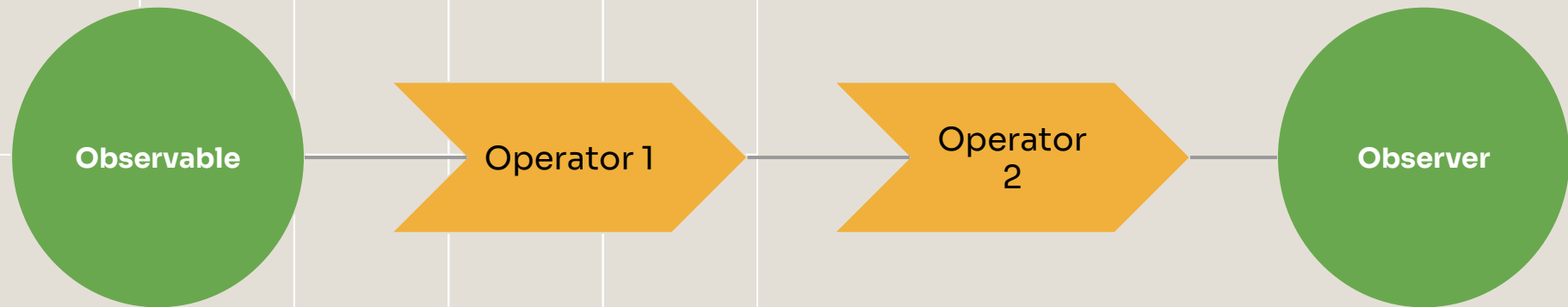
Observable



Observable:

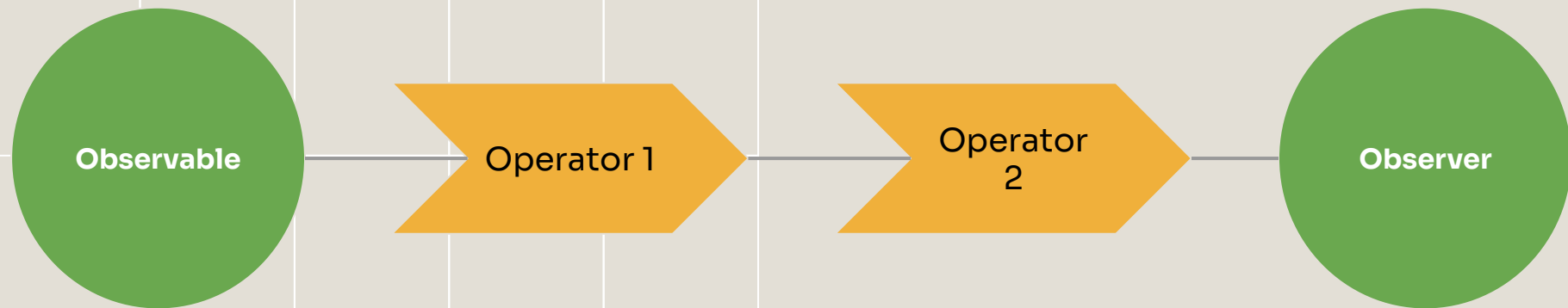
- Produce values
- Has a `Subscribe()` method
- That returns a `Subscription`
- Lazy

Observable



```
ro.NewObservable(func(observer Observer[int]) Teardown {  
    observer.Next(0)  
    observer.Next(1)  
    observer.Complete()  
  
    return func () {  
        println("unsubscribed !")  
    }  
}),
```

Observable



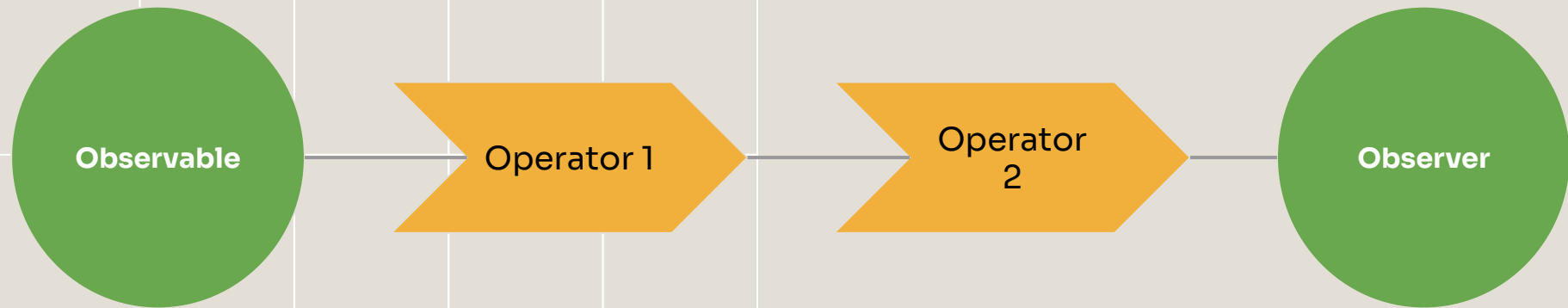
```
observable := ro.Just(1, 2, 3)
```

```
obs := ro.Range(1, 5)
```

```
obs := ro.FromChannel(ch)
```

```
obs := ro.RangeWithInterval(1, 5, 100*time.Millisecond)
```

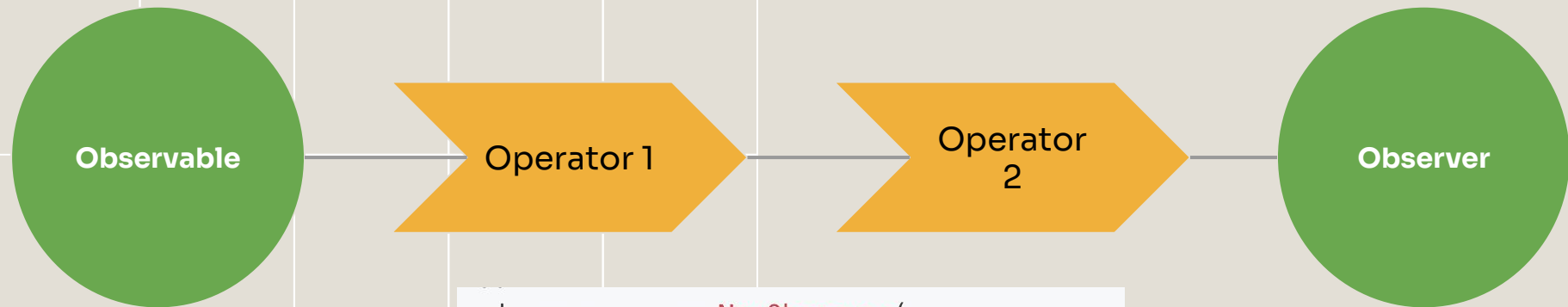
Observer



Observer:

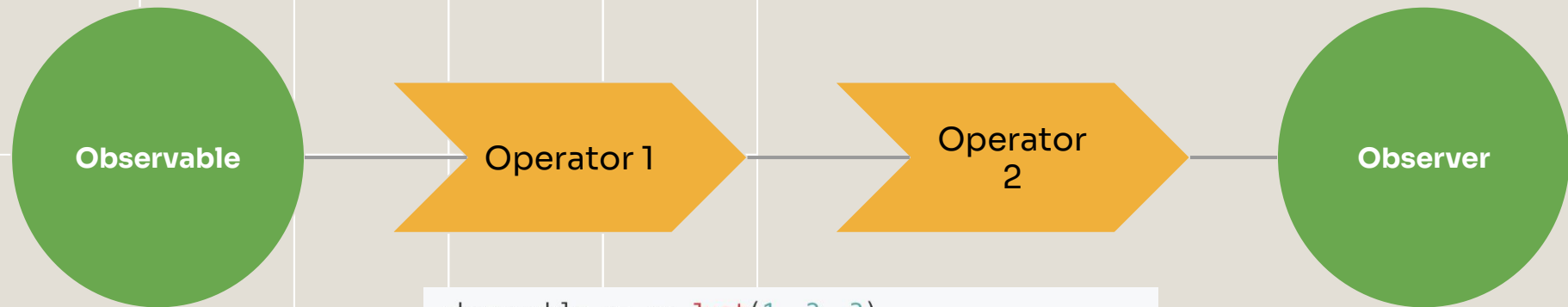
- Receive notifications
- Has Next(), Error(), Complete() methods
- Stateful: active or errored or completed
- Thread-safe

Observer



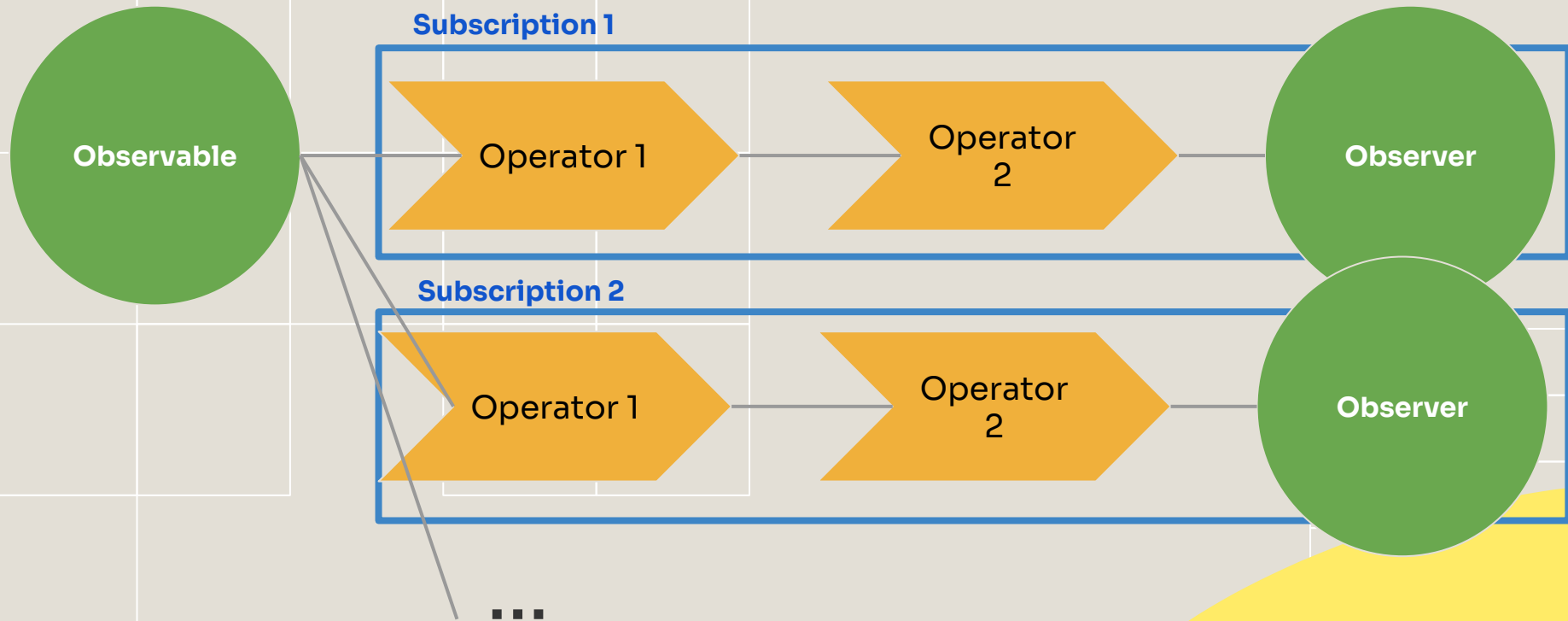
```
observer := ro.NewObserver(  
    func(value int) {  
        fmt.Println("Received:", value)  
    },  
    func(err error) {  
        fmt.Println("Error:", err)  
    },  
    func() {  
        fmt.Println("Completed")  
    },  
)
```


Observer

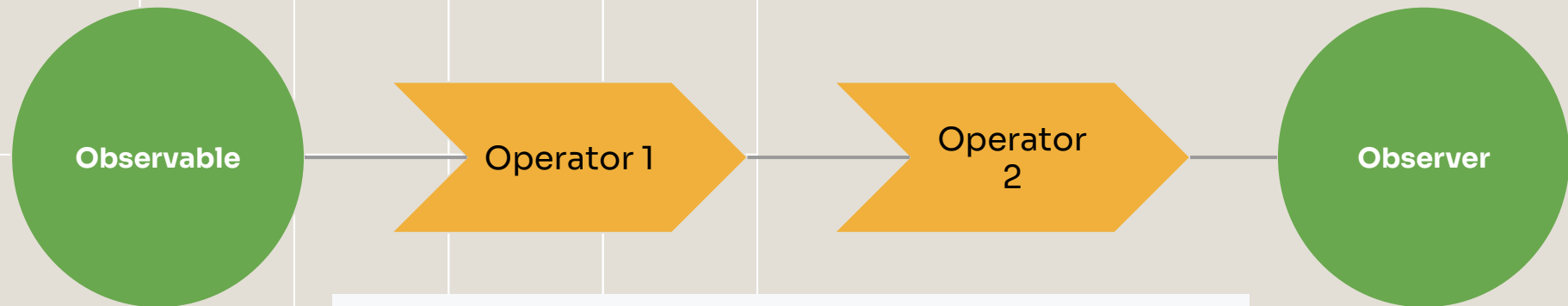


```
observable := ro.Just(1, 2, 3)
observable.Subscribe(ro.NewObserver(
    func(value int) {                // onNext
        fmt.Println("Received:", value)
    },
    func(err error) {                // onError
        fmt.Println("Error:", err)
    },
    func() {                          // onComplete
        fmt.Println("Done!")
    },
))
```

Subscriptions



Subscription

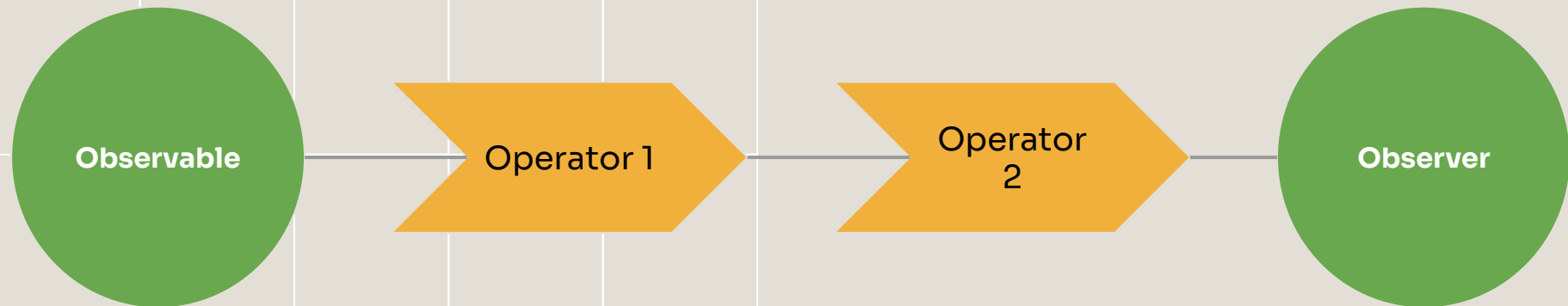


```
observable := ro.Just(1, 2, 3)

// Multiple subscribers
sub1 := observable.Subscribe(ro.OnNext(func(n int) {
    fmt.Println("Subscription 1:", n)
}))

sub2 := observable.Subscribe(ro.OnNext(func(n int) {
    fmt.Println("Subscription 2:", n)
}))
```

Subscription

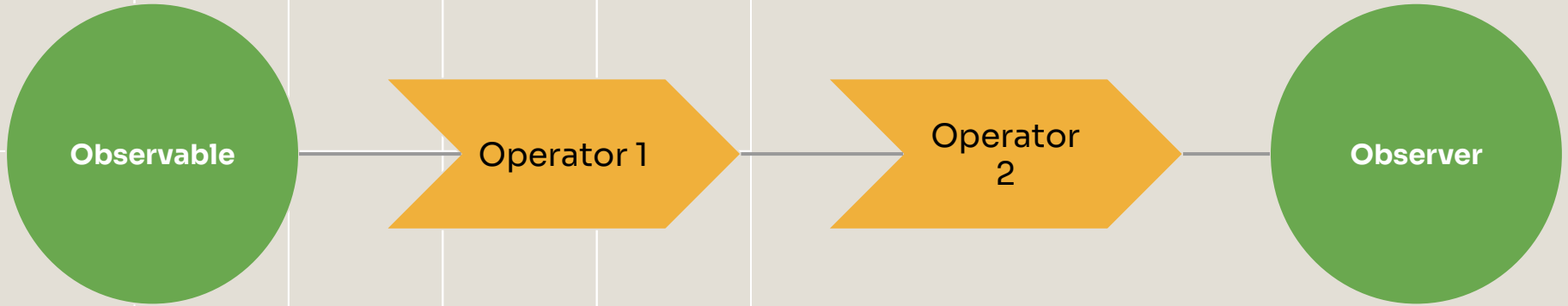


```
// Create an Observable
observable := ro.Interval(1 * time.Second)

// Subscribe and get the subscription
subscription := observable.Subscribe(ro.OnNext(func(tick int64) {
    fmt.Println("Tick:", tick)
})))

// Cancel the subscription
subscription.Unsubscribe()
```

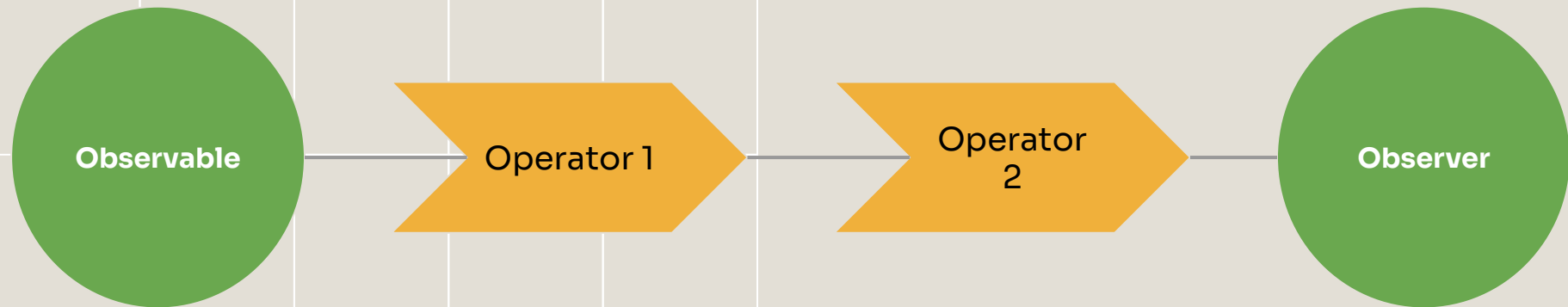
Operator



Operator:

- Has both an Observable and Observer
- Can be stateful

Operator

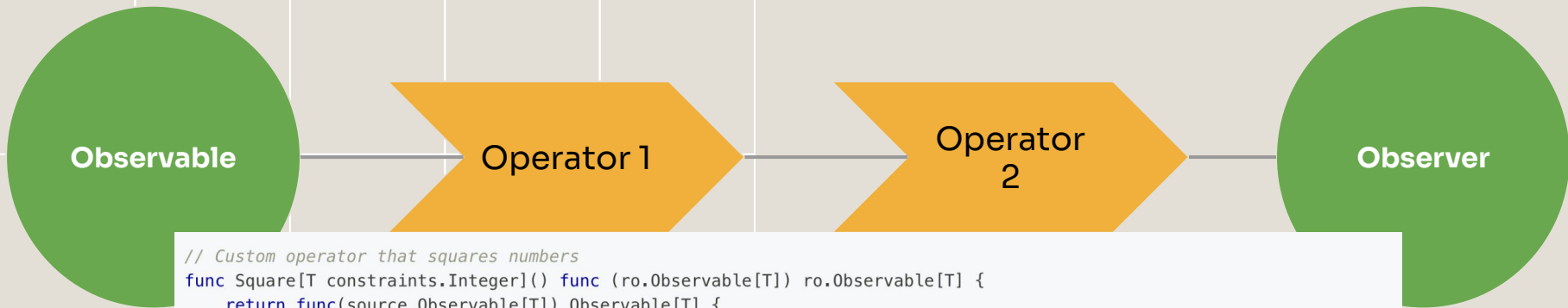


```
ro.Filter(func(x int) bool {  
    return x%2 == 0  
})),
```

```
ro.Map(func(x int) string {  
    return fmt.Sprintf("blabla %d", x)  
})),
```

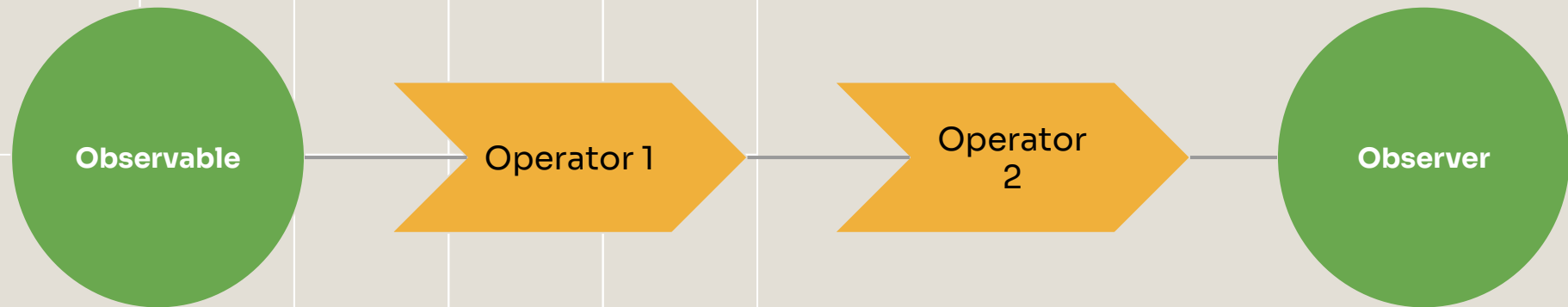
```
ro.Take[int64](5),
```

Operator



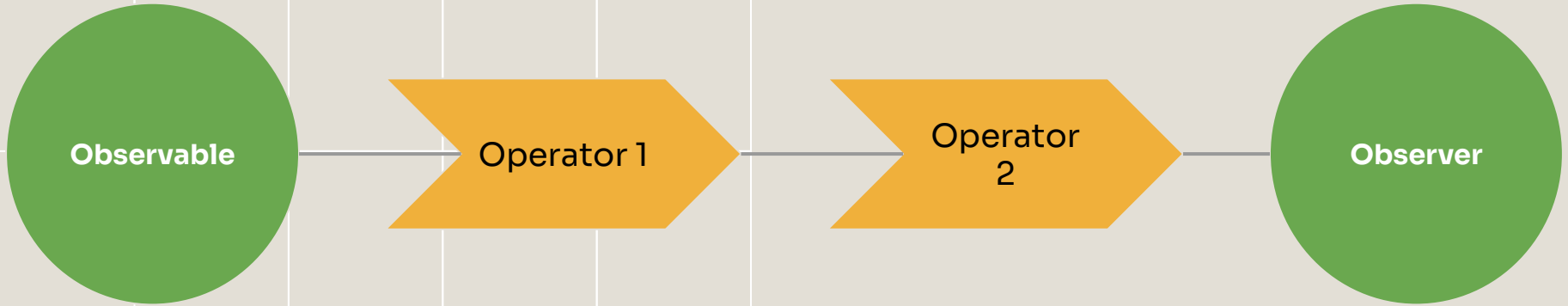
```
// Custom operator that squares numbers
func Square[T constraints.Integer]() func (ro.Observable[T]) ro.Observable[T] {
    return func(source Observable[T]) Observable[T] {
        return NewUnsafeObservableWithContext(func(subscriberCtx context.Context, destination Observer[T]) Teardown {
            sub := source.SubscribeWithContext(
                subscriberCtx,
                NewObserverWithContext(
                    func(ctx context.Context, value T) {
                        destination.NextWithContext(ctx, value*value)
                    },
                    destination.ErrorWithContext,
                    destination.CompleteWithContext,
                ),
            ),
        )
        return sub.Unsubscribe
    })
}
```

Operator

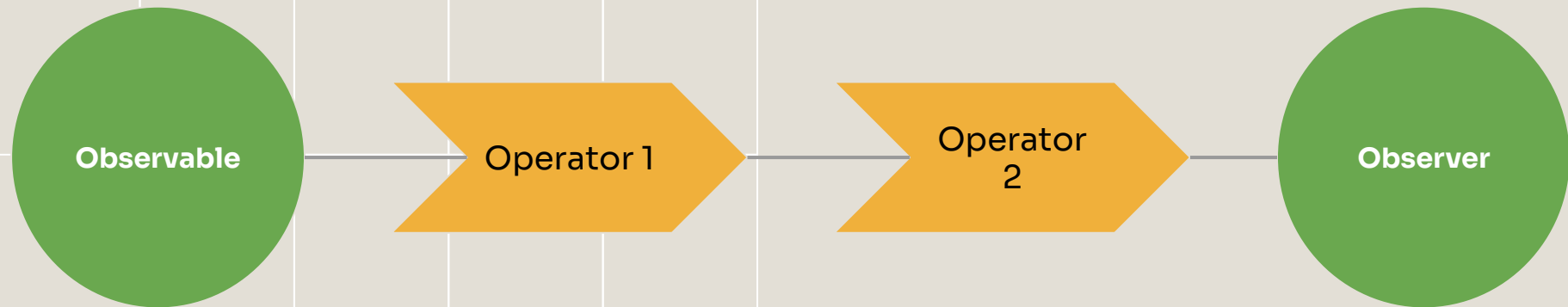


```
observable := ro.Pipe2(  
    ro.Interval(1 * time.Second),  
    ro.Take[int64](5),  
    ro.Map(func(x int64) string {  
        return fmt.Sprintf("Tick: %d", x)  
    }),  
)
```

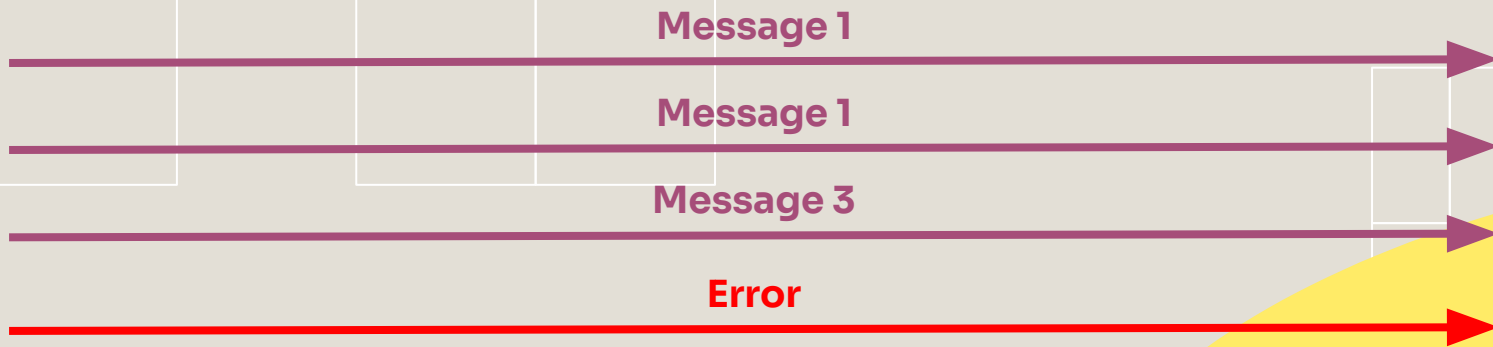
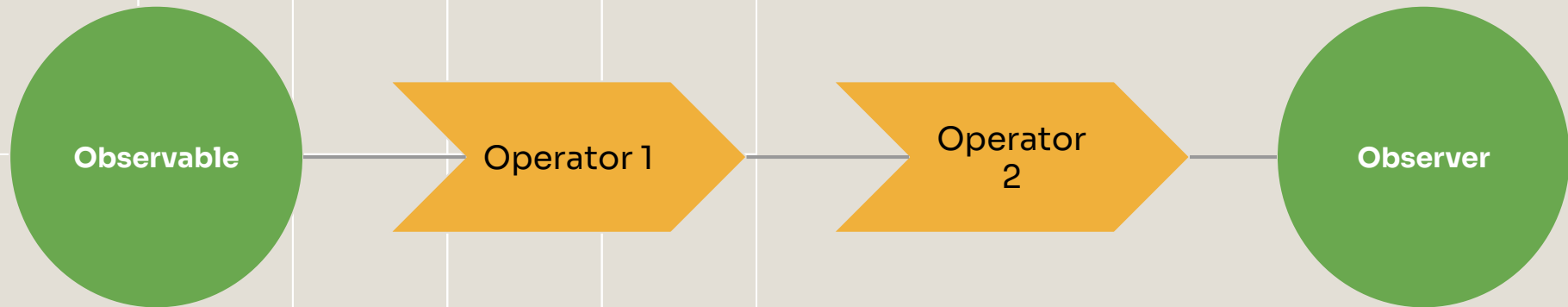

Lifecycle



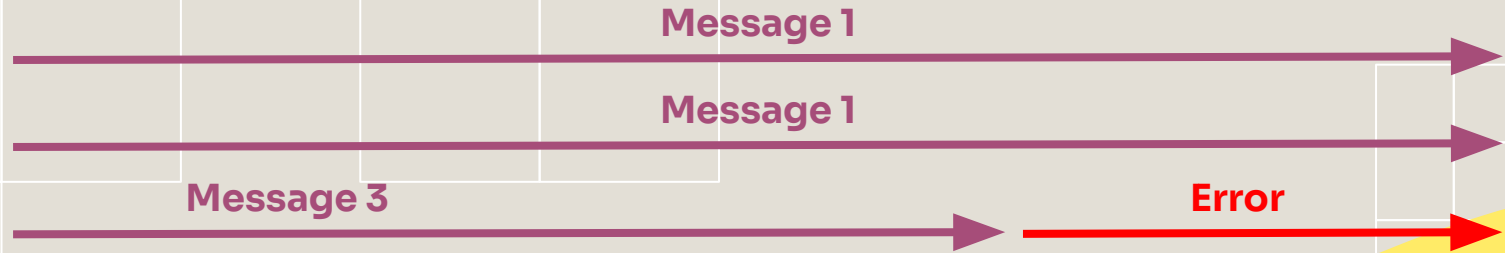
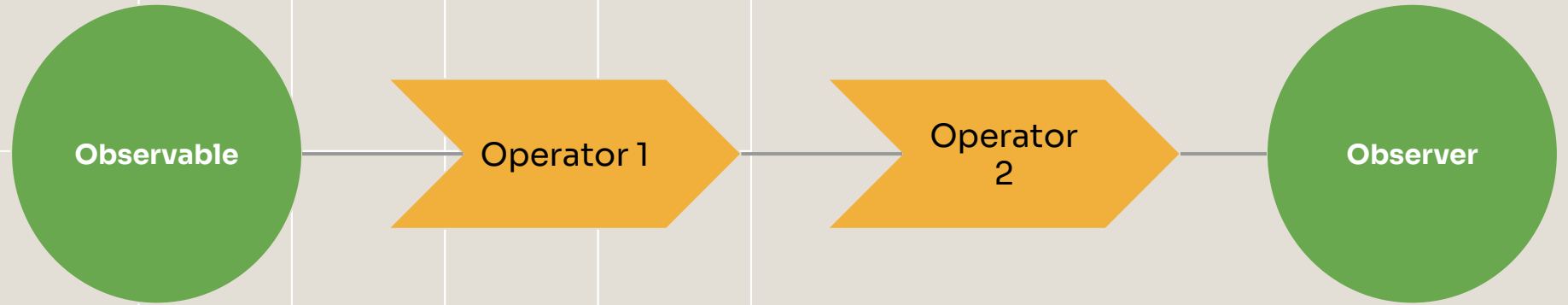
Lifecycle



Lifecycle



Lifecycle



Creating Observables

Operators that originate new Observables.

- **Create** — create an Observable from scratch by calling observer methods programmatically
- **Defer** — do not create the Observable until the observer subscribes, then create an Observable for each observer
- **Empty / Never / Throw** — create Observables that have very precise semantics
- **From** — convert some other object or data structure into an Observable
- **Interval** — create an Observable that emits a sequence of integers at a given interval
- **Just** — convert an object or a set of objects into an Observable that emits them immediately
- **Range** — create an Observable that emits a range of sequential integers
- **Repeat** — create an Observable that emits a particular item or sequence of items repeatedly
- **Start** — create an Observable that emits the return value of a function before emitting the items from the source Observable
- **Timer** — create an Observable that emits a single item after a given delay

Transforming Observables

Operators that transform items that are emitted by an Observable.

- **Buffer** — periodically gather items from an Observable into bundles of a specified size, then emitting the items one at a time
- **FlatMap** — transform the items emitted by an Observable into Observables, then flatten the resulting nested Observable into a single Observable
- **GroupBy** — divide an Observable into a set of Observables that each emit items from the original Observable, organized by key
- **Map** — transform the items emitted by an Observable by applying a function to each item
- **Scan** — apply a function to each item emitted by an Observable, sequentially, to produce a new sequence of values
- **Window** — periodically subdivide items from an Observable into Observables of a specified size, then emitting the items one at a time

Filtering Observables

Operators that selectively emit items from a source Observable.

- **Debounce** — only emit an item from an Observable if a particular timespan has passed without it emitting another item
- **Distinct** — suppress duplicate items emitted by an Observable
- **ElementAt** — emit only item *n* emitted by an Observable
- **Filter** — emit only those items from an Observable that pass a predicate function
- **First** — emit only the first item, or the first item that meets a condition, from an Observable
- **IgnoreElements** — do not emit any items from an Observable but mirror its errors
- **Last** — emit only the last item emitted by an Observable
- **Sample** — emit the most recent item emitted by an Observable within periodic intervals
- **Skip** — suppress the first *n* items emitted by an Observable
- **SkipLast** — suppress the last *n* items emitted by an Observable
- **Take** — emit only the first *n* items emitted by an Observable
- **TakeLast** — emit only the last *n* items emitted by an Observable

Combining Observables

Operators that work with multiple source Observables to create a single Observable.

- **And / Then / When** — combine sets of items emitted by two or more Observables using different scheduling strategies
- **CombineLatest** — when an item is emitted by either of two Observables, combine the items from both Observables via a specified function and emit items based on the function
- **Join** — combine items emitted by two Observables whenever an item from one Observable is emitted during a time window defined according to an item emitted by the other Observable
- **Merge** — combine multiple Observables into one by merging their emissions
- **StartWith** — emit a specified sequence of items before beginning to emit items from the source Observable
- **Switch** — convert an Observable that emits Observables into a single Observable that emits the most-recently-emitted of those Observables
- **Zip** — combine the emissions of multiple Observables together via a specified function to emit single items for each combination based on the results of this function

Error Handling Operators

Operators that help to recover from error notifications from an Observable.

- **Catch** — recover from an `onError` notification by continuing the sequence without error
- **Retry** — if a source Observable sends an `onError` notification, resubscribe to it in the hopes that it will complete without error

Observable Utility Operators

A toolbox of useful Operators for working with Observables.

- **Delay** — shift the emissions from an Observable forward in time by a particular amount
- **Do** — register an action to take upon a variety of Observable lifecycle events
- **Materialize / Dematerialize** — represent both the items emitted and the notifications sent as emitted items, or reverse this process
- **ObserveOn** — specify the scheduler on which an observer will observe this Observable
- **Serialize** — force an Observable to make serialized calls and to be well-behaved
- **Subscribe** — operate upon the emissions and notifications from an Observable
- **SubscribeOn** — specify the scheduler an Observable should use when it is subscribed to
- **TimeInterval** — convert an Observable that emits items into one that emits indications of the amount of time elapsed between those emissions
- **Timeout** — mirror the source Observable, but issue an error notification if a particular period of time elapses without any emitted items
- **Timestamp** — attach a timestamp to each item emitted by an Observable
- **Using** — create a disposable resource that has the same lifespan as the Observable

Conditional and Boolean Operators

Operators that evaluate one or more Observables or items emitted by Observables.

- **All** — determine whether all items emitted by an Observable meet some criteria
- **Amb** — given two or more source Observables, emit all of the items from only the first of these Observables to emit an item
- **Contains** — determine whether an Observable emits a particular item or not
- **DefaultIfEmpty** — emit items from the source Observable, or a default item if the source Observable emits nothing
- **SequenceEqual** — determine whether two Observables emit the same sequence of items
- **SkipUntil** — discard items emitted by an Observable until a second Observable emits an item
- **SkipWhile** — discard items emitted by an Observable until a specified condition becomes false
- **TakeUntil** — discard items emitted by an Observable after a second Observable emits an item or terminates

<https://reactivex.io/documentation/operators.html>

<https://ro.samber.dev/docs/operator/>

Operators



`map(x => 10 * x)`



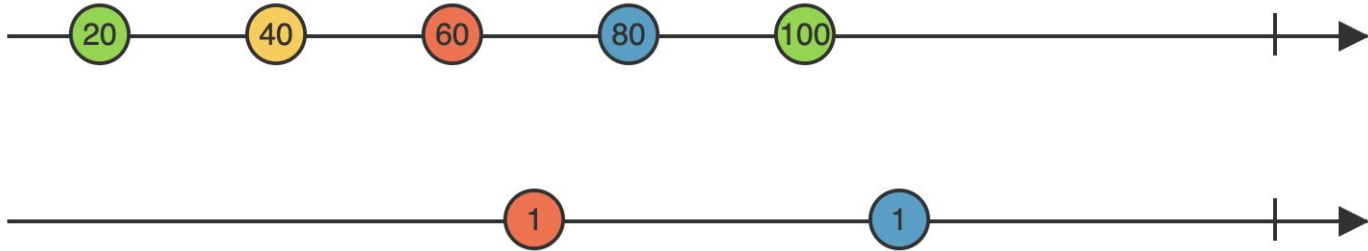
Operators



```
filter(x => x > 10)
```



Operators



merge



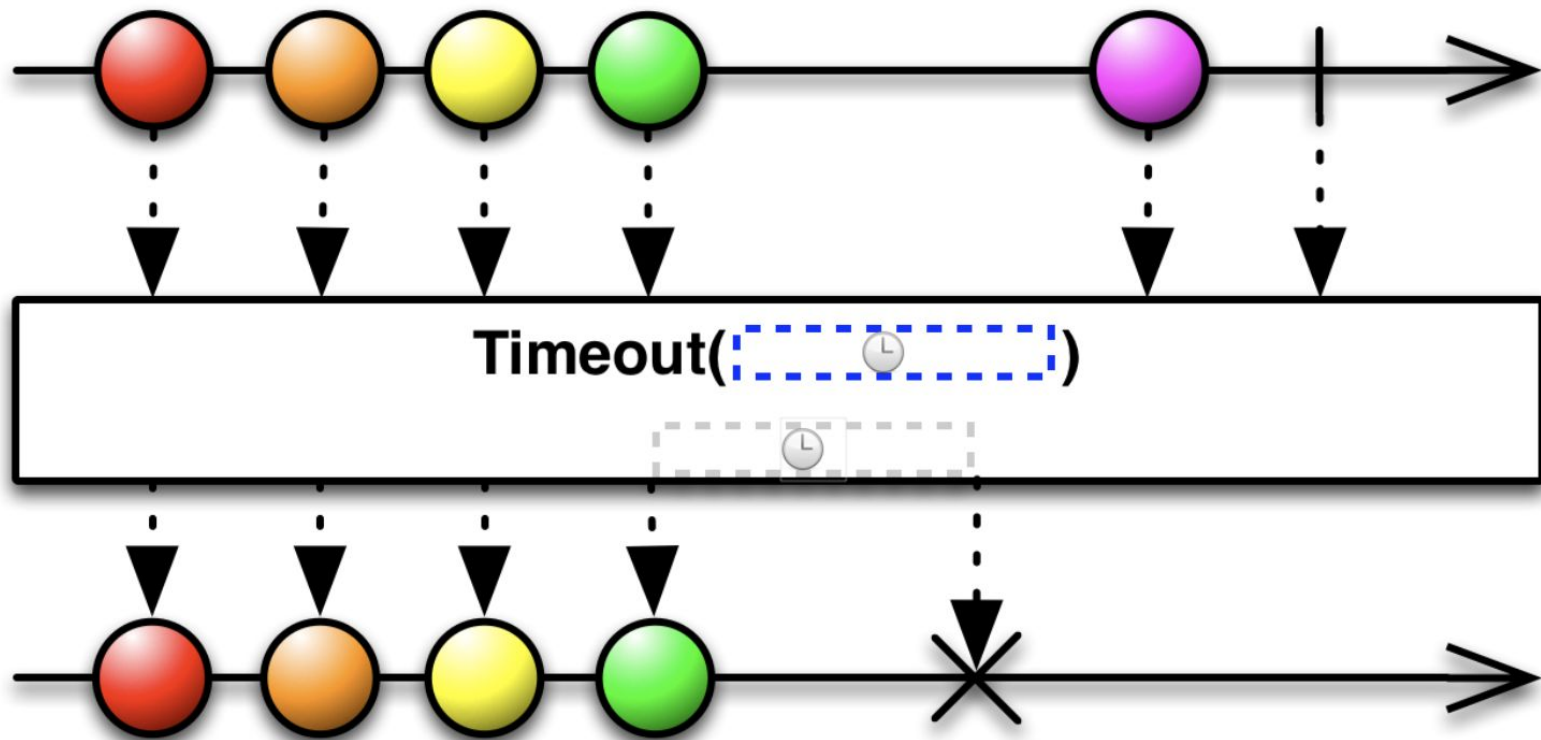
Operators



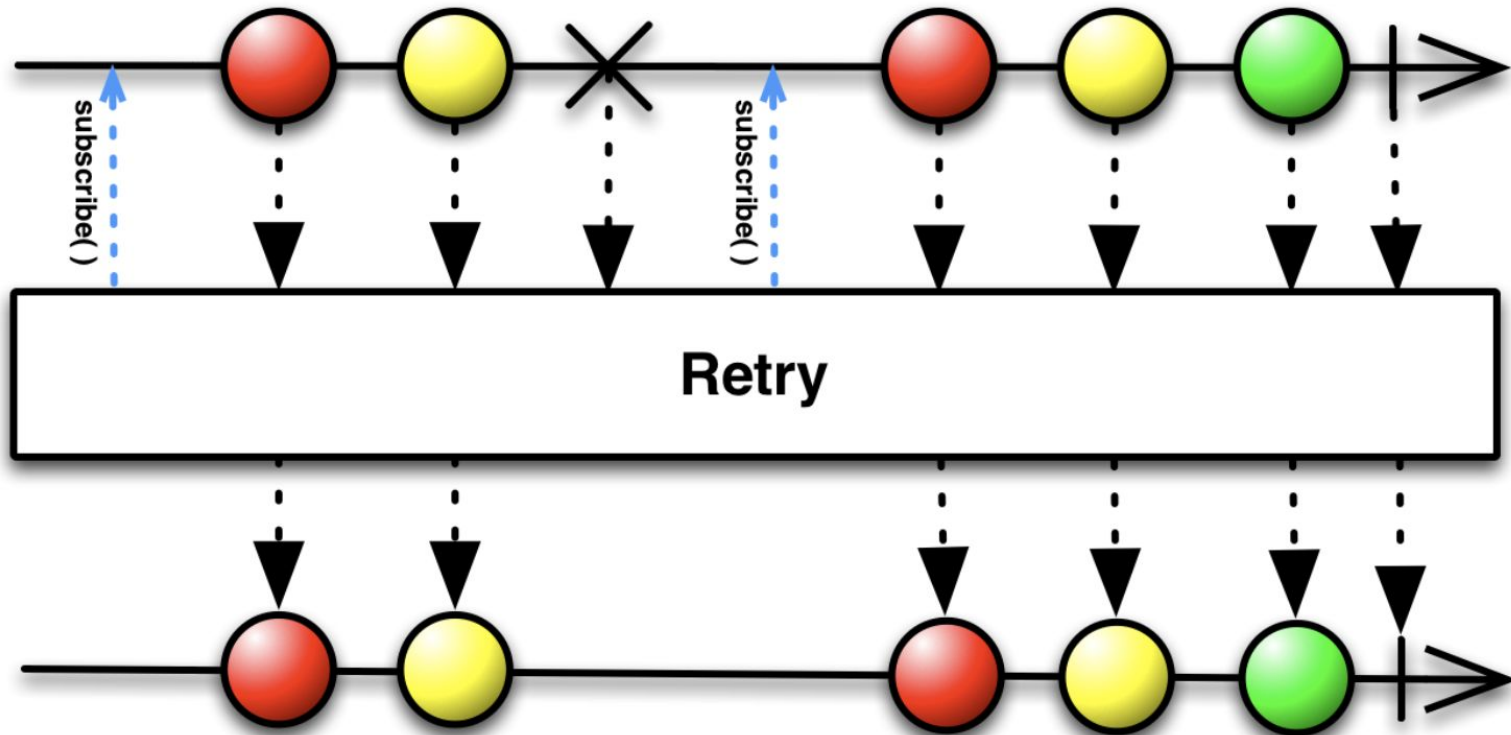
concat



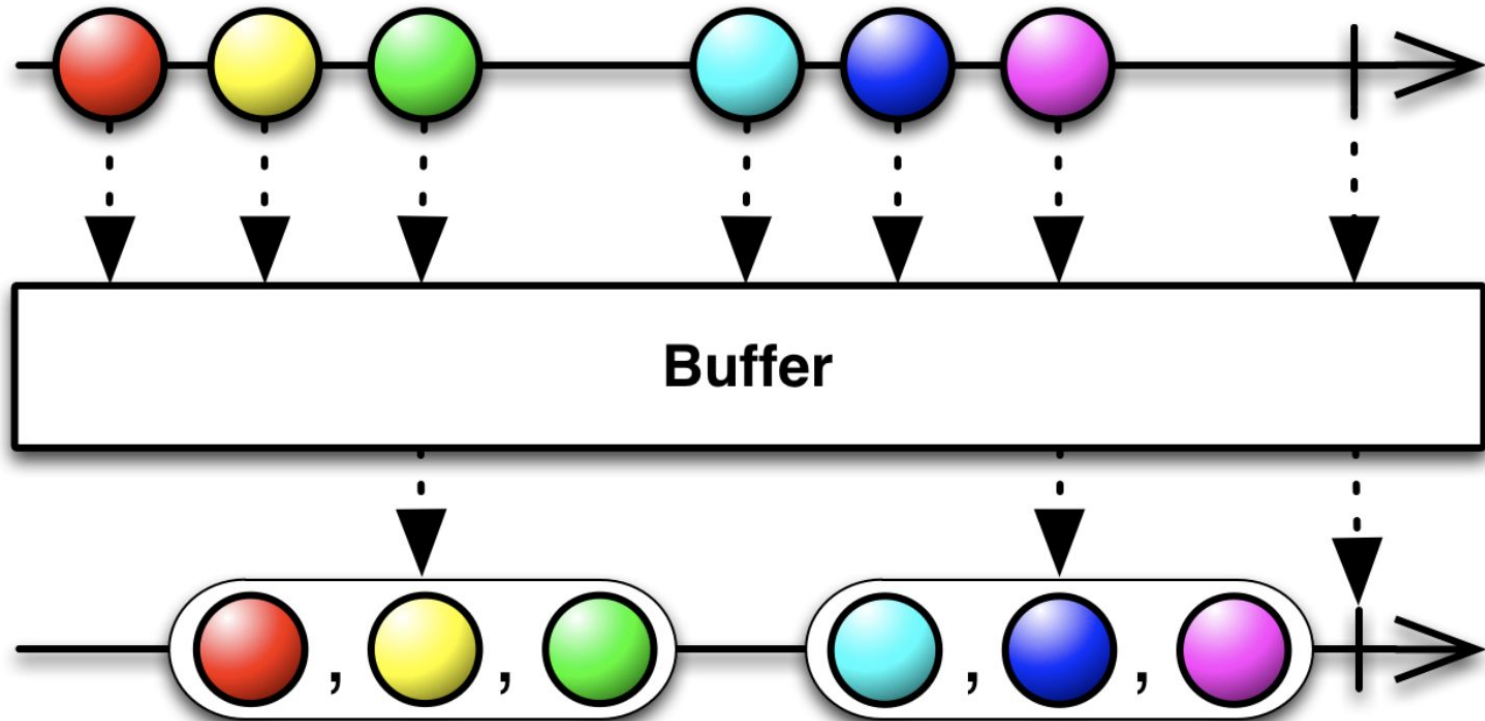
Operators



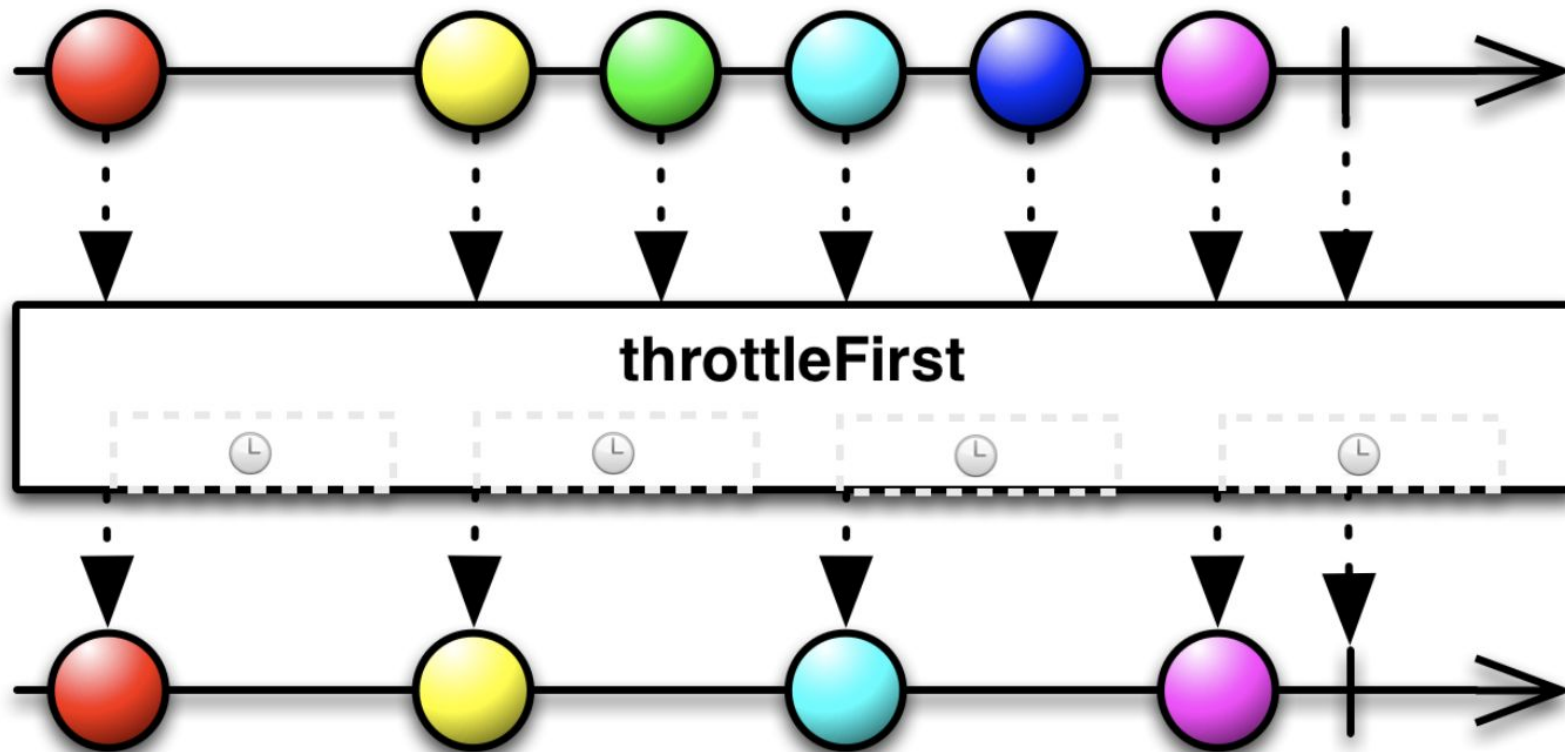
Operators



Operators



Operators



Operators



delay

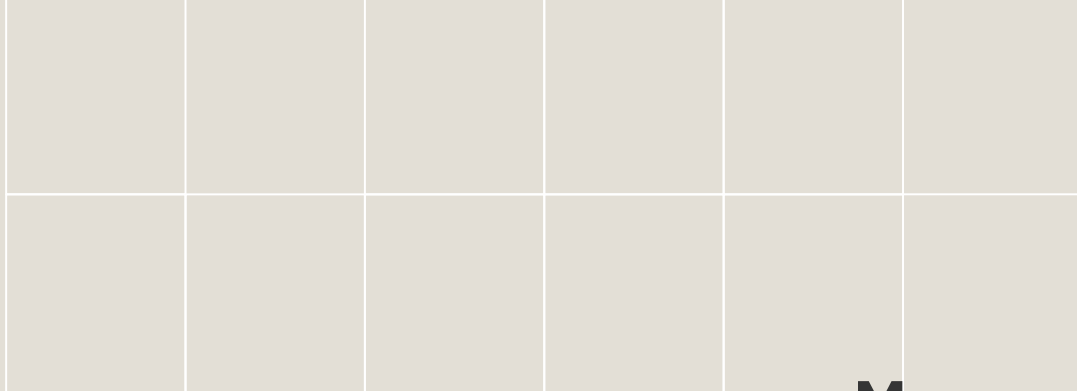


Operators



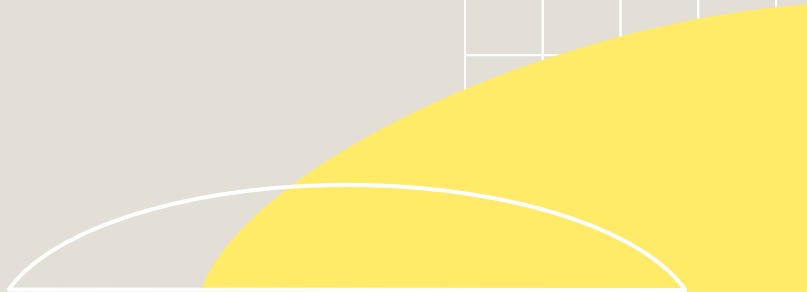
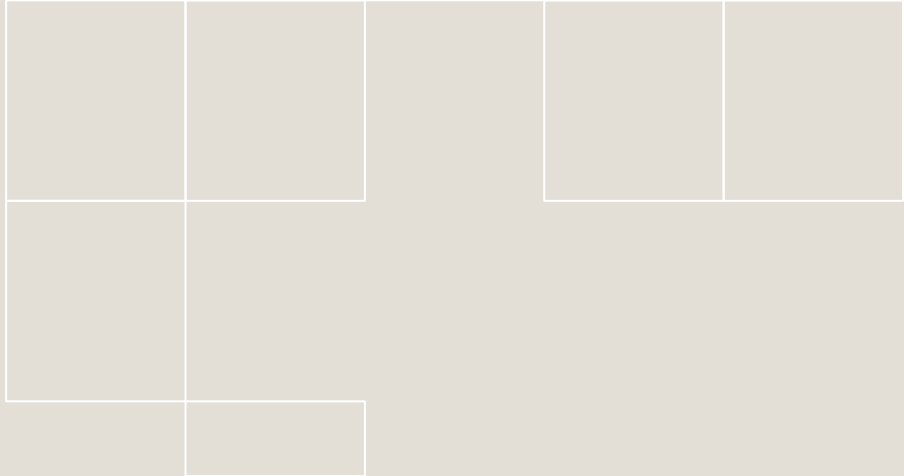
`skipUntil`





More:

<https://reactivex.io/documentation/operators.html>



CSV export

```
54
55 // Define a pipeline to query users from a database and write them to stdout as csv.
56 var pipeline = ro.Pipe7(
57     // Up to 100 page retrieval in database.
58     ro.Range(0, 100),
59     // Query database.
60     // Convert database error into observer.Error() notification.
61     ro.MapErr(getUsers),
62     // Retry on database error.
63     ro.RetryWithConfig([]User)(ro.RetryConfig{
64         MaxRetries: 2,
65         Delay:       5 * time.Second,
66     }),
67     // Complete stream when database response is empty.
68     ro.TakeWhile(func(users []User) bool {
69         return len(users) > 0
70     }),
71     // Convert Observable of []User into Observable of User.
72     ro.Flatten[User](),
73     // Convert User into a csv row ([]string).
74     ro.Map(func(user User) []string {
75         return []string{user.ID, user.Name}
76     }),
77     // Add csv header.
78     ro.StartWith([]string{"ID", "Name"}),
79     // Write csv row to stdout.
80     rocsv.NewCSVWriter(
81         csv.NewWriter(os.Stdout),
82     ),
83 )
84
```

Parallel HTTP Requests

```
// Wait for all requests to complete and combine the results.
var pipeline = ro.Pipe1(
    ro.CombineLatest3(
        // Future is a helper function to run a function in a goroutine.
        // It returns an Observable that emits a single result: a value or an error.
        ro.Future(getOrders),
        ro.Future(getPaymentHistory),
        ro.Future(getUserPreferences),
    ),
    ro.Map(func(values lo.Tuple3[[]string, []string, map[string]any)) result {
        return result{
            Orders:          values.A,
            PaymentHistory: values.B,
            UserPreferences: values.C,
        }
    }),
)
```

Parallel HTTP Requests

```
// Wait for all requests to complete and combine the results.
var pipeline = ro.Pipe1(
    ro.CombineLatest3(
        // Future is a helper function to run a function in a goroutine.
        // It returns an Observable that emits a single result: a value or an error.
        ro.Future(getOrders),
        ro.Future(getPaymentHistory),
        ro.Future(getUserPreferences),
    ),
    ro.Map(func(values lo.Tuple3[[]string, []string, map[string]any)) result {
        return result{
            Orders:          values.A,
            PaymentHistory: values.B,
            UserPreferences: values.C,
        }
    }),
)
```

Tracking stock price

```
57
58 var downstreamPipeline = ro.PipeOp[*TickerData, *Minute](
59     // Prepare the input data
60     ro.Map(func(value *TickerData) []*TickerDataItem {
61         return value.Data
62     }),
63     // Flatten the items from []*TickerDataItem to *TickerDataItem
64     ro.Flatten[*TickerDataItem](),
65     // Split the stream into a stream of streams, one for each symbol
66     ro.GroupBy(func(value *TickerDataItem) string {
67         return value.Symbol
68     }),
69     ro.Map(func(symbolObservable ro.Observable[*TickerDataItem]) ro.Observable[*Minute] {
70         windowPeriod := 1 * time.Minute
71
72         // The first window will end at the end of the current minute, not in 1 minute.
73         // It permits to align window start with the exact clock minutes.
74         now := time.Now()
75         firstWindow := windowPeriod - now.Sub(now.Truncate(windowPeriod))
76
77         // Process the data for each symbol
78         return ro.Pipe[*TickerDataItem, *Minute](
79             symbolObservable,
80             // Buffer 1min of data, to measure the high, low, last price and volume.
81             ro.BufferWhen[*TickerDataItem](ro.IntervalWithInitial(firstWindow, windowPeriod)),
82             ro.Map(func(values []*TickerDataItem) *Minute {
83                 // Measure the high, low, last price and volume
84                 timestamp := time.UnixMilli(values[0].Timestamp).Truncate(time.Minute)
85                 maxPrice := lo.MaxBy(values, func(a, b *TickerDataItem) bool {
86                     return a.Price > b.Price
87                 })
88                 minPrice := lo.MinBy(values, func(a, b *TickerDataItem) bool {
89                     return a.Price < b.Price
90                 })
91                 lastPrice := lo.MaxBy(values, func(a, b *TickerDataItem) bool {
92                     return a.Timestamp > b.Timestamp
93                 })
94                 volume := lo.SumBy(values, func(a *TickerDataItem) float64 {
95                     return a.Volume
96                 })
97
```

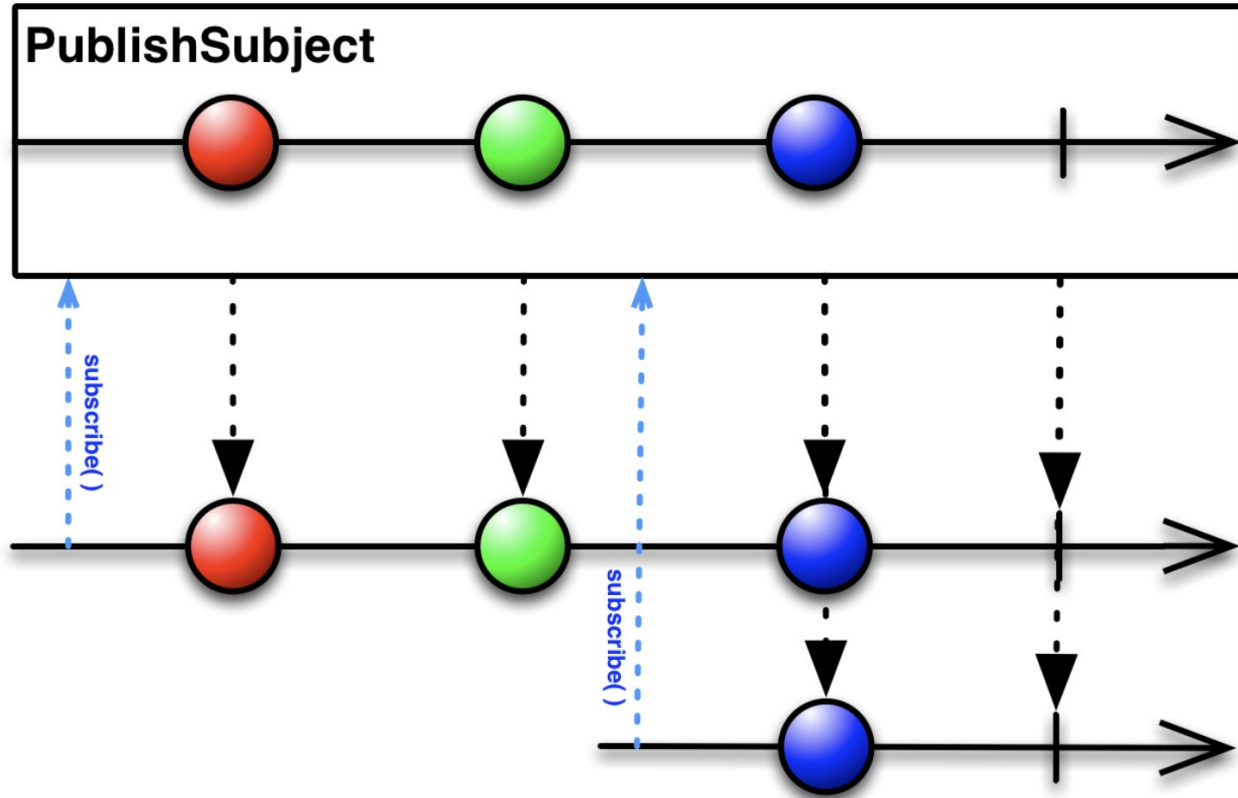
```
97
98     // Return 1min aggregated values for the symbol
99     return &Minute{
100         Symbol: values[0].Symbol,
101         Time:     timestamp,
102         LastPrice: lastPrice.Price,
103         Volume:    volume,
104         High60s:   maxPrice.Price,
105         Low60s:    minPrice.Price,
106     }
107 },
108 )
109 },
110 // Merge all the symbols into a single stream
111 ro.MergeAll[*Minute](),
112 // Adjust precision
113 ro.Map(func(value *Minute) *Minute {
114     truncate := func(value float64) float64 {
115         switch {
116             case value < 1:
117                 return math.Round(value*100000) / 100000
118             case value < 10:
119                 return math.Round(value*10000) / 10000
120             case value < 100:
121                 return math.Round(value*1000) / 1000
122             case value < 1000:
123                 return math.Round(value*100) / 100
124             case value < 10000:
125                 return math.Round(value*10) / 10
126             default:
127                 return math.Round(value)
128         }
129     }
130     value.LastPrice = truncate(value.LastPrice)
131     value.High60s = truncate(value.High60s)
132     value.Low60s = truncate(value.Low60s)
133     value.Volume = truncate(value.Volume)
134     return value
135 },
136 )
137
```

Subject

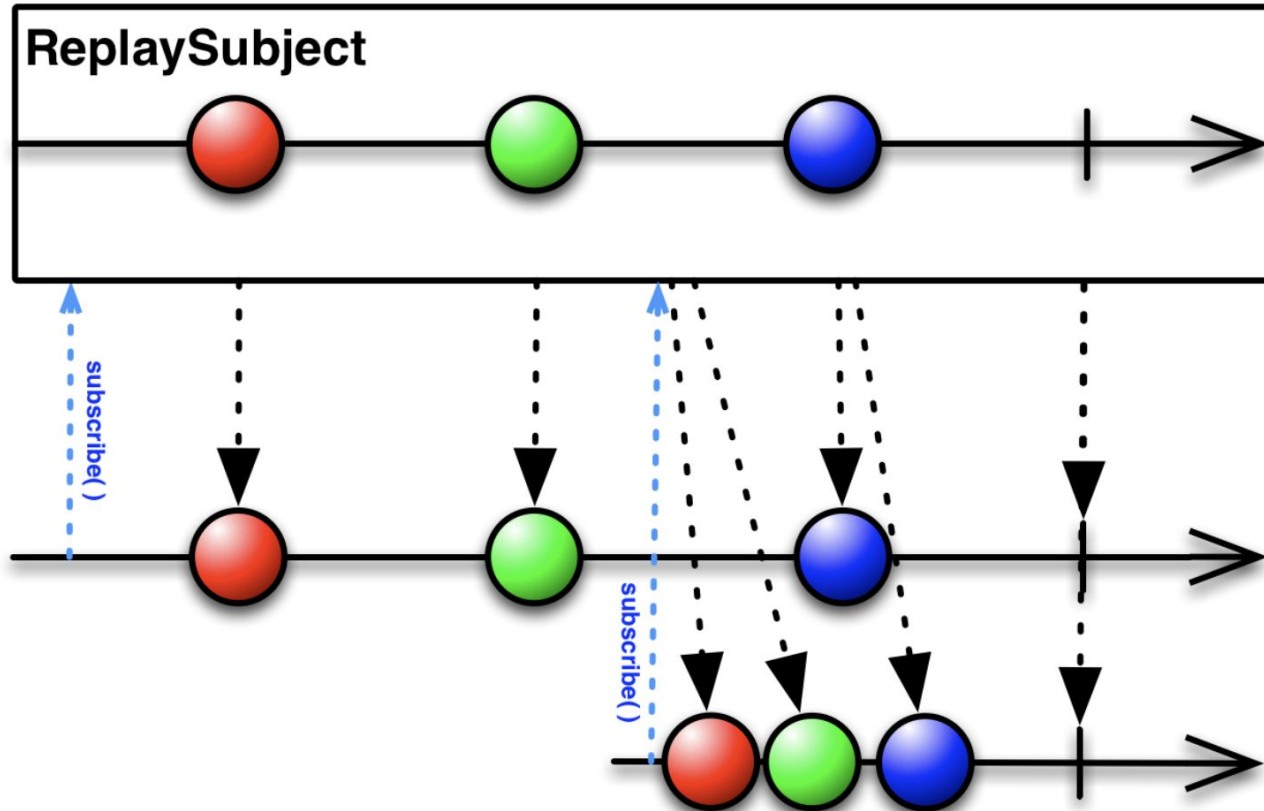
Proxy with:

- single source/upstream subscription
- multiple destination/downstream subscription
- broadcast

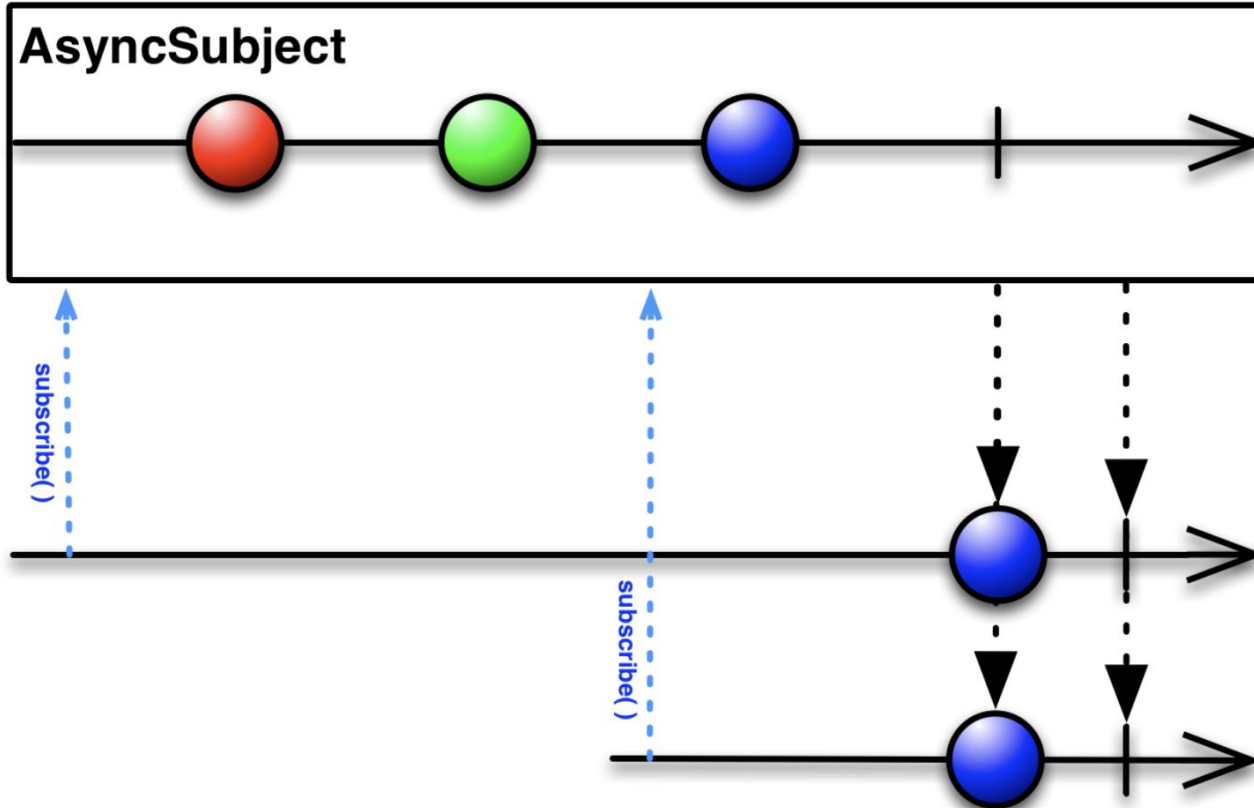
Subject



Subject



Subject




```
// Feature
var pipeline = ro.PipeOp3(
    ro.Filter(func(x int) int {
        return x%2 == 1
    })
    ro.Map(func(x int) string {
        return fmt.Sprintf("processed-%d", x)
    }),
    ro.DelayEach[string](100 * time.Millisecond)
)
```

```
// Tests
func TestObservable(t *testing.T) {
    observable := pipeline(ro.Just(1, 2, 3, 4))

    testing.Assert[string](t).
        Source(observable).
        ExpectNext("processed-1").
        ExpectNext("processed-3").
        ExpectComplete().
        Verify()
}
```

Unit tests

Unit tests

Creating Observables

Operators that originate new Observables.

- **Create** — create an Observable from scratch by calling observer methods programmatically
- **Defer** — do not create the Observable until the observer subscribes, and create a fresh Observable for each observer
- **Empty / Never / Throw** — create Observables that have very precise and limited behavior
- **From** — convert some other object or data structure into an Observable
- **Interval** — create an Observable that emits a sequence of integers spaced by a particular time interval
- **Just** — convert an object or a set of objects into an Observable that emits that or those objects
- **Range** — create an Observable that emits a range of sequential integers
- **Repeat** — create an Observable that emits a particular item or sequence of items repeatedly
- **Start** — create an Observable that emits the return value of a function
- **Timer** — create an Observable that emits a single item after a given delay

Let's go !

github.com/samber/ro



ro - Streams & reactive programming for Go

tag v0.2.0 Go >= 1.18 reference Tests passing go report A+ coverage 68% contributors 4

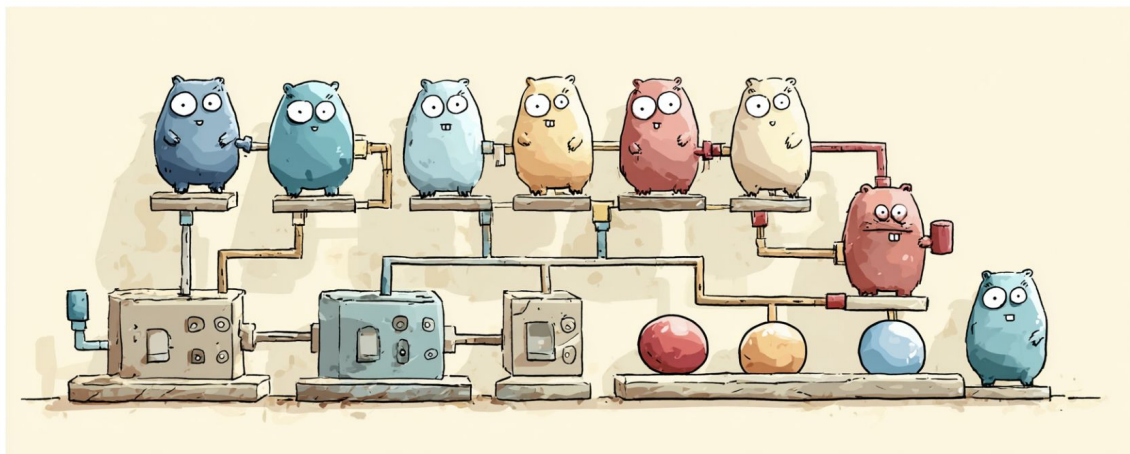
A Go implementation of the [ReactiveX](#) spec.

The purpose of Reactive Programming is to simplify the development of event-driven and asynchronous applications by providing a declarative and composable way to handle streams of data or events.

💖 Support This Project

I'm going all-in on open-source for the coming months.

Help sustain development: Become an [individual sponsor](#) or join as a [corporate sponsor](#).



Plugins

The `ro` library provides a rich ecosystem of plugins for various use cases:

[Full documentation here.](#)

Data Manipulation

- **Bytes** (`plugins/bytes`) - String and byte slice manipulation operators
- **Strings** (`plugins/strings`) - String manipulation operators
- **Sort** (`plugins/sort`) - Sorting operators
- **Type Conversion** (`plugins/strconv`) - String conversion operators

Encoding & Serialization

- **JSON** (`plugins/encoding/json`) - JSON marshaling and unmarshaling
- **CSV** (`plugins/encoding/csv`) - CSV reading and writing
- **Base64** (`plugins/encoding/base64`) - Base64 encoding and decoding
- **Gob** (`plugins/encoding/gob`) - Go binary serialization

Scheduling & Timing

- **Cron** (`plugins/cron`) - Schedule jobs using cron expressions or duration intervals
- **ICS** (`plugins/ics`) - Read and parse ICS/iCal calendars

Network & I/O

- **HTTP** (`plugins/http`) - HTTP request operators
- **I/O** (`plugins/io`) - File and stream I/O operators
- **File System** (`plugins/fsnotify`) - File system monitoring operators

Observability & Logging

- **Log** (`plugins/observability/log`) - Standard logging operators
- **Zap** (`plugins/observability/zap`) - Structured logging with zap
- **Logrus** (`plugins/observability/logrus`) - Structured logging with logrus
- **Slog** (`plugins/observability/slog`) - Structured logging with slog
- **ZeroLog** (`plugins/observability/zerolog`) - Structured logging with zerolog
- **Sentry** (`plugins/observability/sentry`) - Error tracking with Sentry
- **Oops** (`plugins/samber/oops`) - Structured error handling

Rate Limiting

- **Native** (`plugins/ratelimit/native`) - Native rate limiting operators
- **Ulule** (`plugins/ratelimit/ulule`) - Rate limiting with ulule/limiter

Text Processing

- **Regular Expressions** (`plugins/regexp`) - Regular expression operators
- **Templates** (`plugins/template`) - Template processing operators

System Integration

- **Process** (`plugins/proc`) - Process execution operators
- **Signal** (`plugins/signal`) - Signal handling operators
- **Iterators** (`plugins/iter`) - Iterator operators
- **PSI** (`plugins/samber/psi`) - Starvation notifier

Data Validation

- **Validation** (`plugins/ozzo/ozzo-validation`) - Data validation operators

Testing

- **Testing** (`plugins/testify`) - Testing utilities

Utilities

- **HyperLogLog** (`plugins/hyperloglog`) - Cardinality estimation operators
- **Hot** (`plugins/samber/hot`) - In-memory cache

Different concepts

- Arrays/Lists ?
- Iterators/Generators ?
- Future/promise ?
- Other ?

Different concepts

- Kafka Streams
- Spark Streaming
- Flink

Takeaways

Reactive Programming paradigm is:

- Declarative
- Composable
- Abstracted
- Event-driven



Thank you !



github.com/samber/talks



github.com/sponsors/samber



“Comment créer une équipe d'agents IA en 2026”



Benoît Hediard

Founder @Globetrotters.ai | Ex-CTO &
Co-Founder @Agorapulse | Angel Investor

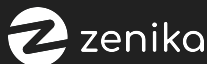


15 janvier 2026 à 19h00



4 Rue Voltaire, 44000 Nantes

chez **_icilundi**



SHIFT TIME TO CUSTOM

27 - 29 Mars
2026

